



## **Atividade somativa – semana 4**

---



# Nome Completo: Marcos Daniel Santana

## Roteiro para realização da atividade somativa

O objetivo desta atividade somativa é fazer com que os dois modelos de redes neurais estudados até aqui (rede neural MLP e rede neural convolucional) identifiquem corretamente todos os números de 0 a 9 dos dois conjuntos de imagens que serão fornecidos. Para isso, você realizará o treinamento e o comparativos de desempenho dos modelos, também fornecidos, alterando parâmetros dos códigos em linguagem Python durante a bateria de treinamentos e testes.

## Carregando os códigos

Inicialmente, baixe para seu computador os arquivos: “S4\_MLP.py” e “S4\_CNN.spy”, disponibilizados.

Abra o Anaconda Navigator e execute o Spyder, clicando em “Launch”, exatamente como você fez nas unidades de estudo anteriores.

No menu “File” da IDE Spyder, selecione “Open” e carregue os arquivos “S4\_MLP.py” e “S4\_CNN.py”.

## Carregando as imagens

Baixe para seu computador a pasta “RN” disponibilizada para *download* no link a seguir:

Esta pasta contém dois conjuntos de imagens de tamanho 28x28 *pixels*, cada qual contendo dez imagens com números de 0 a 9, desenhados manualmente, e que serão utilizados para comparativo de desempenho entre os dois tipos de rede neural utilizados.

IMPORTANTE 1: o primeiro conjunto de dez imagens contém números de 0 a 9 desenhados manualmente, mas foi retirado (copiado) da base de dados utilizada para treinamento dos modelos das redes; o segundo conjunto também contém números de 0 a 9, mas estes foram desenhados pelo professor e não fazem parte da mesma estrutura utilizada nos treinamentos.

IMPORTANTE 2: esta pasta “RN”, contendo as 20 imagens, deverá ser colada na raiz da unidade “C” do seu computador, ficando o caminho da seguinte forma: **C:/RN**

## Bateria de treinamentos e testes

### 1) Primeira parte:

- Nos códigos em Python fornecidos, realize **todas** as alterações possíveis, e que julgar necessárias, nos parâmetros ajustáveis dos dois modelos, de acordo com os conhecimentos adquiridos até agora.



- Para cada ajuste realizado (alteração nos parâmetros), treine os modelos e anote os valores dos ajustes e os valores obtidos para a função de perda e métrica de desempenho para os dois modelos (MLP e CNN), bem como quais foram a função e a métrica utilizadas em cada um deles. **Dica:** crie uma tabela no MS Excel para ajudar-lhe nessa tarefa. Assim, você conseguirá escolher o melhor desempenho ao final da bateria, sabendo quais são os parâmetros necessários para se alcançar tal desempenho.
- Refaça os ajustes e os treinamentos quantas vezes julgar necessário, até que você consiga obter os melhores resultados para os dois modelos.
- Ao final do experimento, quando julgar que os dois modelos estejam suficientemente treinados e apresentando desempenho adequado para a solução do problema, crie uma tabela que contemple os dois melhores modelos escolhidos, contendo: i) **todos** os parâmetros ajustáveis, e as configurações utilizadas; ii) o número previsto pelo modelo em cada uma das dez imagens de teste (de 0 a 9); iii) os valores obtidos para a função de perda e métrica de desempenho.

## 2) Segunda parte:

- Mantenha os dois modelos com os parâmetros ajustados de acordo com os dois melhores desempenhos obtidos na primeira parte desta bateria de treinamento e testes.
- Nos dois códigos Python dos modelos, altere os nomes das imagens de teste, para que eles carreguem o segundo conjunto de imagens. Para isso, basta inserir a letra “b” minúscula após o número, em cada um dos nomes dos arquivos. Ex.: zero0.png deverá ser alterado para zero0b.png; um1.png deverá ser alterado para um1b.png; dois2.png deverá ser alterado para dois2b.png; e assim por diante.
- Execute os dois códigos novamente, para realizar o treinamento das redes e os testes com apresentação dos resultados.
- Crie uma tabela que contemple os dois melhores modelos escolhidos, contendo: i) **todos** os parâmetros ajustáveis, e as configurações utilizadas (devem ser as mesmas informadas na primeira parte); ii) o número previsto pelo modelo em cada uma das dez imagens de teste (de 0 a 9); iii) os valores obtidos para a função de perda e métrica de desempenho.

## 3) Terceira parte (resultados):

- Discorra sua análise num breve parágrafo, sobre as principais características e diferenças de desempenho entre os dois tipos de redes neurais utilizados.
- Os resultados de desempenho da segunda parte do experimento foram os mesmos obtidos na primeira parte? Por quê?
- Os números identificados nas imagens de 0 a 9 no segundo experimento foram os mesmos da primeira parte? Por quê?



## RESULTADOS DA ATIVIDADE SOMATIVA 1

### Primeira parte — melhores modelos obtidos

Os arquivos S4\_MLP.py e S4\_CNN.py foram executados diversas vezes, alterando sistematicamente os parâmetros ajustáveis de cada modelo (funções de ativação, função de perda, otimizador, métrica, número de camadas e neurônios, percentual de dados de treinamento/teste, número de épocas e tamanho do lote), até se chegar às configurações abaixo, escolhidas por apresentarem o melhor desempenho (menor função de perda e maior acurácia) dentre as combinações testadas.

#### Rede Neural MLP — melhor configuração (arquivo S4\_MLP.py)

| Parâmetro                                | Valor / Configuração utilizada |
|------------------------------------------|--------------------------------|
| Camada oculta 1                          | 256 neurônios, ativação tanh   |
| Camada oculta 2                          | 128 neurônios, ativação tanh   |
| Camada de saída                          | 10 neurônios, ativação softmax |
| Função de perda                          | categorical_crossentropy       |
| Otimizador                               | adam                           |
| Métrica de desempenho                    | accuracy                       |
| Percentual de dados de teste             | 20% (80% para treinamento)     |
| Número de épocas                         | 15                             |
| Número de amostras (batch size)          | 128                            |
| <b>Previsões nas 10 imagens de teste</b> |                                |
| zero0.png (real = 0)                     | previsto = 0 (correto)         |
| um1.png (real = 1)                       | previsto = 1 (correto)         |



|                                            |                        |
|--------------------------------------------|------------------------|
| dois2.png (real = 2)                       | previsto = 2 (correto) |
| tres3.png (real = 3)                       | previsto = 3 (correto) |
| quatro4.png (real = 4)                     | previsto = 4 (correto) |
| cinco5.png (real = 5)                      | previsto = 5 (correto) |
| seis6.png (real = 6)                       | previsto = 6 (correto) |
| sete7.png (real = 7)                       | previsto = 7 (correto) |
| oito8.png (real = 8)                       | previsto = 8 (correto) |
| nove9.png (real = 9)                       | previsto = 9 (correto) |
| Função de perda (categorical_crossentropy) | 0.1059                 |
| Métrica de desempenho (accuracy)           | 0.9716                 |

#### Rede Neural Convolucional — melhor configuração (arquivo S4\_CNN.py)

| Parâmetro                              | Valor / Configuração utilizada |
|----------------------------------------|--------------------------------|
| Percentual de dados de treinamento     | 40%                            |
| Matriz de convolução (Camada 1 e 2)    | 3x3                            |
| Nº de filtros - Camada de Convolução 1 | 32                             |
| Nº de filtros - Camada de Convolução 2 | 64                             |
| Nº de neurônios - Camada Densa Oculta  | 128                            |



|                                          |                          |
|------------------------------------------|--------------------------|
| Ativação (Conv1 / Conv2 / Densa Oculta)  | relu / relu / relu       |
| Ativação da camada de saída              | softmax                  |
| Otimizador                               | adam                     |
| Função de perda                          | categorical_crossentropy |
| Métrica de desempenho                    | accuracy                 |
| Número de épocas                         | 5                        |
| Número de amostras (batch size)          | 128                      |
| <b>Previsões nas 10 imagens de teste</b> |                          |
| zero0.png (real = 0)                     | previsto = 0 (correto)   |
| um1.png (real = 1)                       | previsto = 1 (correto)   |
| dois2.png (real = 2)                     | previsto = 2 (correto)   |
| tres3.png (real = 3)                     | previsto = 3 (correto)   |
| quatro4.png (real = 4)                   | previsto = 4 (correto)   |
| cinco5.png (real = 5)                    | previsto = 5 (correto)   |
| seis6.png (real = 6)                     | previsto = 6 (correto)   |
| sete7.png (real = 7)                     | previsto = 7 (correto)   |
| oito8.png (real = 8)                     | previsto = 8 (correto)   |
| nove9.png (real = 9)                     | previsto = 9 (correto)   |



|                                            |        |
|--------------------------------------------|--------|
| Função de perda (categorical_crossentropy) | 0.0578 |
| Métrica de desempenho (accuracy)           | 0.9819 |

## Segunda parte — mesmos parâmetros, segundo conjunto de imagens

Mantendo exatamente os mesmos parâmetros ajustáveis definidos na primeira parte, os nomes das imagens de teste foram alterados para o segundo conjunto (sufixo "b", desenhado pelo professor) nos arquivos S4\_MLP\_partB.py e S4\_CNN\_partB.py, e os modelos foram treinados e testados novamente.

### Rede Neural MLP — segundo conjunto de imagens (arquivo S4\_MLP\_partB.py)

| Parâmetro                                | Valor / Configuração utilizada |
|------------------------------------------|--------------------------------|
| Camada oculta 1                          | 256 neurônios, ativação tanh   |
| Camada oculta 2                          | 128 neurônios, ativação tanh   |
| Camada de saída                          | 10 neurônios, ativação softmax |
| Função de perda                          | categorical_crossentropy       |
| Otimizador                               | adam                           |
| Métrica de desempenho                    | accuracy                       |
| Percentual de dados de teste             | 20% (80% para treinamento)     |
| Número de épocas                         | 15                             |
| Número de amostras (batch size)          | 128                            |
| <b>Previsões nas 10 imagens de teste</b> |                                |
| zero0b.png (real = 0)                    | previsto = 0 (correto)         |



|                                            |                        |
|--------------------------------------------|------------------------|
| um1b.png (real = 1)                        | previsto = 2 (ERRADO)  |
| dois2b.png (real = 2)                      | previsto = 2 (correto) |
| tres3b.png (real = 3)                      | previsto = 2 (ERRADO)  |
| quatro4b.png (real = 4)                    | previsto = 7 (ERRADO)  |
| cinco5b.png (real = 5)                     | previsto = 5 (correto) |
| seis6b.png (real = 6)                      | previsto = 5 (ERRADO)  |
| sete7b.png (real = 7)                      | previsto = 2 (ERRADO)  |
| oito8b.png (real = 8)                      | previsto = 3 (ERRADO)  |
| nove9b.png (real = 9)                      | previsto = 3 (ERRADO)  |
| Função de perda (categorical_crossentropy) | 0.1059                 |
| Métrica de desempenho (accuracy)           | 0.9716                 |

Rede Neural Convolucional — segundo conjunto de imagens (arquivo S4\_CNN\_partB.py)

| Parâmetro                              | Valor / Configuração utilizada |
|----------------------------------------|--------------------------------|
| Percentual de dados de treinamento     | 40%                            |
| Matriz de convolução (Camada 1 e 2)    | 3x3                            |
| Nº de filtros - Camada de Convolução 1 | 32                             |
| Nº de filtros - Camada de Convolução 2 | 64                             |



|                                          |                          |
|------------------------------------------|--------------------------|
| Nº de neurônios - Camada Densa Oculta    | 128                      |
| Ativação (Conv1 / Conv2 / Densa Oculta)  | relu / relu / relu       |
| Ativação da camada de saída              | softmax                  |
| Otimizador                               | adam                     |
| Função de perda                          | categorical_crossentropy |
| Métrica de desempenho                    | accuracy                 |
| Número de épocas                         | 5                        |
| Número de amostras (batch size)          | 128                      |
| <b>Previsões nas 10 imagens de teste</b> |                          |
| zero0b.png (real = 0)                    | previsto = 9 (ERRADO)    |
| um1b.png (real = 1)                      | previsto = 1 (correto)   |
| dois2b.png (real = 2)                    | previsto = 2 (correto)   |
| tres3b.png (real = 3)                    | previsto = 3 (correto)   |
| quatro4b.png (real = 4)                  | previsto = 4 (correto)   |
| cinco5b.png (real = 5)                   | previsto = 5 (correto)   |
| seis6b.png (real = 6)                    | previsto = 5 (ERRADO)    |
| sete7b.png (real = 7)                    | previsto = 2 (ERRADO)    |
| oito8b.png (real = 8)                    | previsto = 8 (correto)   |



|                                            |                       |
|--------------------------------------------|-----------------------|
| nove9b.png (real = 9)                      | previsto = 3 (ERRADO) |
| Função de perda (categorical_crossentropy) | 0.0578                |
| Métrica de desempenho (accuracy)           | 0.9819                |

### Terceira parte — análise dos resultados

Os dois modelos foram capazes de aprender a tarefa de classificação de dígitos com bom desempenho sobre o conjunto de teste do MNIST, mas com diferenças relevantes entre si. A rede MLP, por tratar a imagem como um vetor de 784 valores independentes (sem considerar a estrutura espacial dos pixels), obteve acurácia de 97,16% no conjunto de teste. Já a CNN, que preserva a estrutura espacial da imagem por meio das camadas de convolução e pooling, alcançou uma acurácia superior, de 98,19%, com uma função de perda também menor (0,0578 contra 0,1059 da MLP). Essa diferença ficou ainda mais evidente ao testar os modelos com o segundo conjunto de imagens (sufixo "b"), desenhado pelo professor fora do padrão do MNIST: a MLP acertou apenas 3 dos 10 dígitos, enquanto a CNN acertou 6 dos 10. Isso confirma o que a teoria prevê: a capacidade da CNN de extrair características (bordas, curvas, texturas) de forma parcialmente independente da posição exata dos traços na imagem faz com que ela generalize melhor para variações de estilo de escrita do que a MLP, que é sensível a pequenos deslocamentos e distorções nos pixels de entrada.

Os resultados de desempenho (função de perda e acurácia) obtidos na segunda parte do experimento foram numericamente idênticos aos da primeira parte, tanto para a MLP (perda 0,1059 / acurácia 97,16%) quanto para a CNN (perda 0,0578 / acurácia 98,19%). Isso ocorreu por dois motivos: primeiro, essas métricas são calculadas sobre o conjunto de teste do próprio MNIST, e não sobre as imagens da pasta "RN" (que são usadas apenas para a previsão qualitativa exibida no gráfico); segundo, nos códigos utilizados foi fixada uma semente aleatória (random seed = 42) antes do treinamento, o que torna a inicialização dos pesos sinápticos e a divisão dos dados totalmente reprodutíveis — repetir o treinamento com os mesmos parâmetros e a mesma semente produz exatamente o mesmo resultado. Caso a semente aleatória não fosse fixada, seria esperado observar pequenas variações entre as duas execuções, pois cada novo treinamento parte de pesos inicializados aleatoriamente e o gradiente descendente é sensível ao ponto de partida no espaço de busca, podendo convergir para um mínimo ligeiramente diferente da função de perda.

Já os números identificados nas imagens de 0 a 9 não foram os mesmos entre as duas partes. Enquanto os dois modelos classificaram corretamente os 10 dígitos do primeiro conjunto de imagens, o desempenho caiu de forma acentuada no segundo conjunto: a MLP passou de 10/10 para 3/10 acertos, e a CNN, de 10/10 para 6/10. A explicação está na origem das imagens: o primeiro conjunto foi retirado da própria base MNIST (isto é, pertence à mesma distribuição de dados vista durante o treinamento), enquanto o segundo



conjunto foi desenhado manualmente pelo professor, com traços, espessura, centralização e estilo de escrita diferentes dos exemplos de treinamento. Trata-se de um caso típico de mudança de distribuição entre os dados de treinamento e os dados de uso real do modelo: por melhor que seja a acurácia obtida em um conjunto de teste extraído da mesma fonte dos dados de treino, isso não garante o mesmo desempenho sobre dados provenientes de uma fonte diferente. A CNN generalizou melhor que a MLP nesse cenário porque suas camadas convolucionais aprendem características locais (bordas, curvas) que são parcialmente invariantes a pequenas mudanças de posição, espessura e estilo do traço, enquanto a MLP aprende diretamente a partir dos valores brutos de cada pixel, tornando-a mais sensível a esse tipo de variação.



## Experimento complementar — Data Augmentation (aumento de dados)

Os resultados da segunda parte mostraram que ambos os modelos, embora bem treinados, perdem desempenho quando testados com dígitos escritos fora do padrão da base MNIST. Para investigar se essa limitação poderia ser reduzida sem alterar a arquitetura nem os demais parâmetros ajustáveis, foi realizado um experimento complementar aplicando data augmentation (aumento de dados) ao treinamento dos dois modelos. Os arquivos utilizados foram S4\_MLP\_aug.py e S4\_CNN\_aug.py.

A técnica consiste em, ao invés de apresentar sempre a mesma imagem à rede, transformar aleatoriamente cada imagem do conjunto de treinamento a cada época — girando, deslocando e ampliando/reduzindo o dígito dentro de limites definidos. Foram aplicadas três transformações:

| Transformação     | Parâmetro | Efeito sobre a imagem                                        |
|-------------------|-----------|--------------------------------------------------------------|
| RandomRotation    | 0,08      | gira a imagem em até $\pm 29$ graus                          |
| RandomTranslation | 0,10      | desloca em até 10% da altura/largura ( $\approx 2,8$ pixels) |
| RandomZoom        | 0,10      | amplia ou reduz o dígito em até 10%                          |

Essas camadas ficam ativas apenas durante o treinamento; na fase de previsão o Keras as desativa automaticamente, de modo que a imagem de teste chega intacta ao modelo. Como a rede nunca vê duas vezes exatamente a mesma imagem, o aprendizado por época torna-se mais lento, e por isso o número de épocas foi triplicado (MLP: 15  $\rightarrow$  45; CNN: 5  $\rightarrow$  15). Todos os demais parâmetros ajustáveis foram mantidos idênticos aos das partes anteriores, de modo que a única variável alterada entre as duas colunas do comparativo a seguir é a presença do aumento de dados.

### Comparativo de desempenho: sem e com data augmentation

| Indicador                     | MLP sem aug. | MLP com aug. | CNN sem aug. | CNN com aug. |
|-------------------------------|--------------|--------------|--------------|--------------|
| Função de perda (teste MNIST) | 0,1059       | 0,0562       | 0,0578       | 0,0514       |
| Acurácia (teste MNIST)        | 97,16%       | 98,27%       | 98,19%       | 98,54%       |



|                                  |       |       |       |       |
|----------------------------------|-------|-------|-------|-------|
| Acertos — conjunto 1 (MNIST)     | 10/10 | 10/10 | 10/10 | 10/10 |
| Acertos — conjunto 2 (professor) | 3/10  | 9/10  | 6/10  | 9/10  |

### Previsões detalhadas no segundo conjunto de imagens

| Modelo       | 0        | 1        | 2 | 3        | 4        | 5 | 6        | 7        | 8        | 9        | Acertos |
|--------------|----------|----------|---|----------|----------|---|----------|----------|----------|----------|---------|
| MLP sem aug. | 0        | <b>2</b> | 2 | <b>2</b> | <b>7</b> | 5 | <b>5</b> | <b>2</b> | <b>3</b> | <b>3</b> | 3/10    |
| MLP com aug. | 0        | 1        | 2 | 3        | 4        | 5 | <b>5</b> | 7        | 8        | 9        | 9/10    |
| CNN sem aug. | <b>9</b> | 1        | 2 | 3        | 4        | 5 | <b>5</b> | <b>2</b> | 8        | <b>3</b> | 6/10    |
| CNN com aug. | 0        | 1        | 2 | 3        | 4        | 5 | <b>5</b> | 7        | 8        | 9        | 9/10    |

Na tabela acima, a primeira linha de cabeçalho indica o dígito real de cada imagem, e os valores em negrito assinalam as previsões incorretas.

### Análise do experimento complementar

O aumento de dados produziu ganho expressivo no segundo conjunto de imagens: a MLP passou de 3/10 para 9/10 acertos e a CNN, de 6/10 para 9/10, sem que nenhum dos modelos perdesse desempenho no primeiro conjunto (ambos mantiveram 10/10). A melhoria também apareceu na métrica calculada sobre o conjunto de teste do MNIST, com queda da função de perda e aumento da acurácia nos dois modelos, o



que indica que a técnica atuou como um regularizador, e não apenas como um ajuste específico para as imagens do professor.

A razão do ganho está diretamente ligada à causa da queda de desempenho identificada na segunda parte. Na base MNIST, todo dígito é normalizado para caber em uma caixa de 20x20 pixels centralizada pelo centro de massa, enquanto os dígitos desenhados pelo professor apresentam variação de tamanho, de espessura de traço e de centralização. Ao ser treinada com deslocamentos, rotações e zoom aleatórios, a rede deixa de depender daquele enquadramento rígido e passa a reconhecer o dígito mesmo quando ele aparece deslocado ou em escala diferente. Ou seja, o aumento de dados resolve, do lado do treinamento, boa parte do problema de padronização das imagens de entrada — sem que tenha sido necessário alterar o pré-processamento aplicado às imagens de teste.

Um caso particularmente ilustrativo é o dígito 7 do segundo conjunto, desenhado no estilo europeu, com um traço horizontal cortando o meio da haste diagonal. Esse formato é raro na base MNIST, composta majoritariamente por escrita de estudantes norte-americanos, na qual o 7 costuma aparecer sem o corte. Sem aumento de dados, os dois modelos classificaram esse dígito como 2 — o traço horizontal superior somado ao corte central produz um conjunto de traços que, para uma rede que nunca viu um 7 cortado, se aproxima mais de um 2. Com o aumento de dados, ambos os modelos passaram a classificá-lo corretamente: ao serem expostos a versões giradas e deslocadas dos dígitos durante o treinamento, os modelos aprendem a reconhecer o 7 pela relação geral entre a haste horizontal superior e a diagonal descendente, tornando-se menos dependentes da presença ou ausência de um traço específico.

O único dígito que permaneceu incorreto nas quatro configurações testadas foi o 6 do segundo conjunto, sempre classificado como 5. Diferentemente do caso do 7, aqui o problema não é de enquadramento nem de espessura do traço, mas de forma: o 6 desenhado apresenta a curva superior bastante aberta e alongada, aproximando-se visualmente de um 5. Nenhuma das transformações geométricas aplicadas (rotação, deslocamento ou zoom) modifica essa característica, razão pela qual o erro persiste mesmo com o aumento de dados.

Por fim, o comparativo evidencia a diferença estrutural entre as duas arquiteturas. Sem aumento de dados, a CNN generalizou consideravelmente melhor que a MLP para os dígitos do professor (6/10 contra 3/10). Isso decorre de a CNN, por meio das camadas de convolução, aprender a compor características locais — bordas, curvas e cantos — que permanecem reconhecíveis mesmo quando o traço muda de posição ou de espessura, enquanto a MLP recebe a imagem como um vetor de 784 pixels tratados de forma independente, sendo por isso diretamente sensível a qualquer deslocamento dos traços. Com o aumento de dados, contudo, a MLP alcançou o mesmo número de acertos da CNN neste conjunto de dez imagens (9/10), justamente por passar a ser treinada com as variações de posição e escala que antes não conhecia. Vale observar que a CNN manteve desempenho superior na métrica calculada sobre o conjunto de teste do MNIST (98,54% contra 98,27%), indicador estatisticamente mais confiável por ser apurado sobre dezenas de milhares de imagens, e não sobre apenas dez.